

Browser Agent Cheat Sheet

Amaç: Bu dokümana bakarak yeni bir web görevi için sıfırdan Browser Agent kur, görevi doğru tasarla, agent'i çalıştır ve sonucu doğrula.

0. Önce şunu anla: Browser Agent nedir?

Browser Agent, bir LLM'e browser kullanma yeteneği verilmiş halidir. LLM ne yapılacağına karar verir; Browser Use sayfaı görmesini ve browser action'larını yapmasını sağlar.

Goal → Observe → Decide → Click / Type / Scroll → Observe again → Verify → Done

- LLM = beyin. Sıradaki adımı seçer.
- Browser Use = LLM ile browser arasındaki köprü/harness.
- Browser = agent'in kullandığı pencere

1. Her görevde önce Browser Agent gerekli mi karar ver

- Web sitesinde gezinme, form doldurma, bilgi toplama veya UI üzerinden tekrarlı (repetitive) işlem varsa uygundur.
- Aynı işi yapan düzgün bir API varsa önce API'yi tercih et.
- Para gönderme, silme, satın alma gibi riskli işlemlerde insan onayı ekle.

2. İlk gün kurulumu - sadece bir kez

Python ortamını yönetmek için uv kur. Aşağıdaki command'lerden bilgisayarına uygun olanı terminal'de run'la!

Mac / Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows - PowerShell

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Terminali kapatıp yeniden aç. Kontrol:

```
uv --version
```

3. Proje klasörü + GitHub + Claude Code

1. VS Code → Open Folder → Desktop → Exposure AI Academy → yeni proje klasörü oluştur.
2. Trust Folder & Continue seç.
3. GitHub → Repositories → New ile boş repo oluştur; GitHub'ın verdiği komutlarla lokal klasörü bağla.
4. VS Code terminalinde Claude Code'u başlat.

```
claude
```

Claude kodu yazmana yardım edecek; Goal, Input, Permissions ve Success Condition'ı sen belirleyeceksin.

4. Projeye özel Python ortamı oluştur

```
uv venv --python 3.12
```

Mac / Linux

```
source .venv/bin/activate
```

Windows

```
.venv\Scripts\activate
```

Yeni terminal açarsan environment'ı tekrar aktif et.

5. Browser Use'u kur

```
uv pip install browser-use==0.13.7 python-dotenv
```

Academy setup'ında Browser Use 0.13.7 kullanıyoruz. Playwright ayrıca kurma; bu çalışan setup'ta Browser Use kendi browser altyapısını yönetir.

6. API key'i güvenli ekle

Proje root'unda .env oluştur ve aşağıdaki key'leri ekle (whatsapp grubunda seninle paylaşıldı):

```
BEDROCK_API_KEY=...  
BEDROCK_BASE_URL=https://bedrock-mantle.us-east-1.api.aws/openai/v1
```

Model: google.gemma-4-31b

.gitignore içine ise aşağıdakileri ekle:

```
.env  
venv/  
__pycache__/  
*.pyc
```

7. Kod yazmadan önce görevi tasarla

Her yeni task'ta önce aşağıdaki 9 soruyu cevapla. Bunlar agent'in gerçek tasarımıdır; kod bundan sonra gelir.

Soru	Kendine sor	Örnek Cevap (agent labs challenge 3)
1. Goal	Agent işin sonunda hangi sonucu elde etmeli?	Student Portal'daki 10 iş başvurusunun tamamını başarıyla göndermeli.
2. Start	Hangi site/sayfadan başlayacak?	student.exposureai.org ana sayfasından başlayacak.
3. Input	Hangi bilgileri kullanacak? profile.md, CSV, metin vb.	Öğrencinin adı, okulu, GitHub'ı, ilgi alanları vb. bilgiler bizim oluşturacağımız job_profile.md içinde yer alacak.
4. Login	Login gerekli mi? İnsan mı yapacak?	Login'i öğrenci Magic Link ile manuel yapacak. Login bittikten sonra agent aynı browser session ile devam edecek.
5. Navigation	Hangi domain'de kalacak? Görünen linklerle mi ilerleyecek?	Sadece student.exposureai.org içinde kalmalı. Internal URL tahmin etmek yerine ekrandaki link ve butonlarla ilerlemeli.
6. Tools	click, input, scroll, dropdown... hangileri gerçekten lazım? (Örnek aksiyonlar: navigate · click · input · send_keys · select_dropdown · scroll · extract · search_page · find_text · screenshot · wait · switch · go_back · upload_file · done)	click, input, scroll, select_dropdown, screenshot, done. Dosya upload veya JavaScript çalıştırma gerekmiyor.
7. Yasaklar	Neyi ASLA yapmamalı? Uydurma, reset, silme, dış site vb.	Profilde olmayan bilgi uydurmamalı, Testi sıfırlaya basmamalı, başka siteme gitmemeli, başka öğrencinin verisine dokunmamalı.
8. Success	Hangi açık text/state görürürse görev tamamdır?	Sayfada Challenge Complete ✓ ve 10 / 10 Applications Submitted görmeden görevi tamamlandı saymamalı.
9. Recovery	Yarıda kalırsa nereden devam edecek?	Önce mevcut durumu tekrar gözlemlemeli. Daha önce Completed olan başvuruları atlayıp yalnızca kalanlardan devam etmeli.

8. Claude Code'a yukarıdaki cevaplarını kullanarak bir prompt ver.

Student Portal üzerinde bir Browser Agent oluşturmak istiyorum.

GOAL: Agent, Student Portal'daki 10 iş başvurusunun tamamını başarıyla göndermeli. START: student.exposureai.org ana sayfasından başlamalı. INPUT: Başvurularda kullanılacak öğrenci bilgileri job_profile.md dosyasında bulunuyor. Agent yalnızca burada bulunan bilgileri kullanmalı. LOGIN: Login'i ben Magic Link ile manuel olarak yapacağım. Login tamamlandıktan sonra agent aynı browser session ile devam etmeli. NAVIGATION: Agent sadece student.exposureai.org domain'i içinde kalmalı. Internal URL'leri tahmin etmemeli; sayfadaki görünen link ve butonlarla ilerlemeli. TOOLS: Görev için gerekli browser aksiyonlarını kullanmalı: click, input, scroll, select_dropdown, screenshot ve done. Gereksiz yetkiler açılmamalı. YASAKLAR: - job_profile.md içinde olmayan bilgileri uydurma. - Testi sıfırla butonuna basma. - Başka domain'e gitme. - Başka öğrencilerin verilerine dokunma. SUCCESS: Görev yalnızca sayfada: "Challenge Complete ✓" ve "10 / 10 Applications Submitted" mesajları görüldüğünde tamamlanmış sayılmalı.

RECOVERY: Bir aksiyon başarısız olursa mevcut sayfayı tekrar gözlemle ve devam etmeyi dene. Agent yarıda kalırsa daha önce Completed olan başvuruları tekrar yapma; yalnızca tamamlanmamış başvurulardan devam et.

9. Claude Code’a agent kodunu yazdır

8. adımda hazırladığın prompt’u Claude Code’a gönder ve Browser Agent’ı kodlamasını iste.

- Claude projeyi anlaşılır dosyalara bölsün ve her dosyanın ne işe yaradığını sana kısaca açıklasın.
- Goal, Input, Browser, Tools ve Success Condition’ın kodda nerede olduğunu anlayabilmelisin.

10. Önce browser’ın çalıştığını kontrol et

Claude’dan görünür Chromium açıp example.com’a giden küçük bir smoke_test.py hazırlamasını iste ve aşağıdaki command’l terminalde run’la:

```
python smoke_test.py
```

Browser açılmıyorsa terminaldeki hatayı Claude Code’a gönderip önce bunu çöz.

11. LLM’i bağla ve küçük bir test yap

Claude, Gemma 4 31B’i .env içindeki BEDROCK_API_KEY ve BEDROCK_BASE_URL ile Browser Use’a bağlasın.

```
Model: google.gemma-4-31b | headless=False | max_actions_per_step=1
```

12. Input’u Claude ile birlikte hazırla

7. adımda belirlediğin Input’u Claude’a anlat ve bu task için agent’ın hangi bilgilere ihtiyaç duyduğunu sor.

- Claude ile profile.md, job_profile.md, CSV vb. uygun input dosyasını oluştur.
- Sadece gerekli bilgileri ekle; input’ta olmayan bilgiyi agent uydurmasın.

13. Login gerekiyorsa sen yap

Browser açılır → sen login olursun → portal açılır → terminalde Enter’a basarsın → agent aynı session ile devam eder.

Login sonrası agent’ı gerekli domain ile sınırla.

14. Gerçek agent’ı çalıştır

Yeni terminal açıtıysan önce .venv’i aktif et, sonra agent dosyasını aşağıdaki command’l terminalde run’layarak çalıştır:

```
Mac / Linux: source .venv/bin/activate
Windows: .venv\Scripts\activate
Run: python agent.py
```

Login dışında browser’a elle yardım etme.

15. Terminalden agent’ı takip et

Terminalde step, action ve error loglarını izle. Agent’ın nerede takıldığını buradan anlayacaksın.

16. Hatanın nerede olduğunu bul

Browser, API, navigation, input veya Success Condition: sorun hangi aşamada çıktı? Terminaldeki hata mesajını not et.

17. Hata varsa Claude Code ile çöz

Terminal logunu Claude Code’a gönder; hatanın nedenini açıklamasını ve gerekli düzeltmeyi yapmasını iste. Sonra agent’ı tekrar çalıştır.

Davranış hatasıysa 7. adımdaki 9 sorudan hangisinin eksik olduğunu kontrol edip prompt’un o kısmını düzelt.

18. Uzun task’larda Recovery ekle

Task yarıda kalırsa Completed işleri atlasın ve sadece kalanlardan devam etsin.

Bir action hata verirse bütün task’ı resetlemek yerine sayfayı tekrar gözlemleyip aynı yerden devam etsin.

19. Çalışan sürümü Git’e kaydet

```
git add .
git commit -m "browser agent çalışıyor"
git push origin main
```

Önemli çalışan aşamalardan sonra commit/push yap.

Amaç: Browser Agent yapmak = hazır Claude prompt’u kopyalamak değildir. Task’ı Goal + Input + Tools + Permissions + Session + Verification + Recovery olarak sen tasarla; Claude Code bu tasarımı koda döksün.